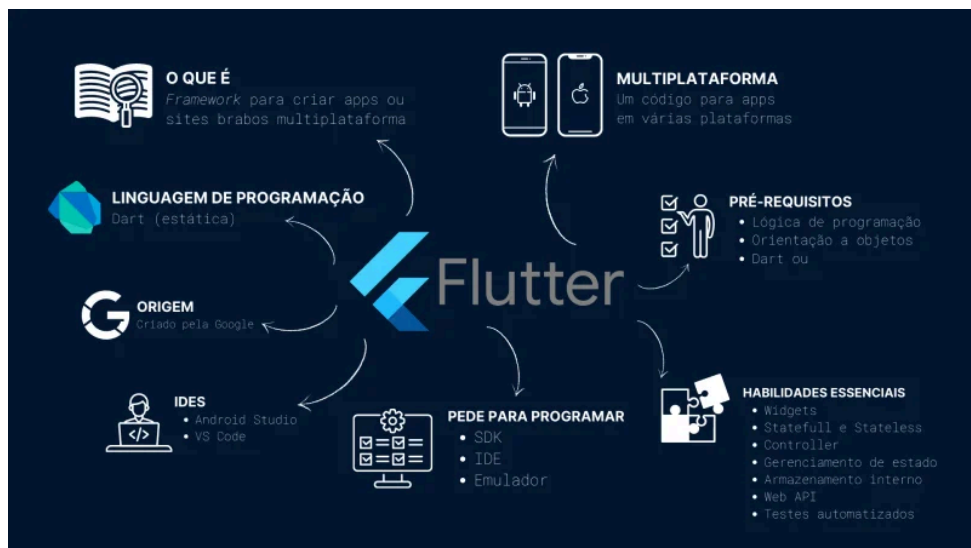


Aula 1 - Meu primeiro App com Flutter

Objetivo da Aula:

- Introdução ao FlutLab
- 'Buildando' uma aplicação Flutter
- Baixar e testar um .apk no smartphone (Android e IOS)
- Programando em Dart
- Manipulando um Widget
- Importando uma biblioteca no Dart
- Customizando nossa primeira aplicação mobile
- Exercícios
- Código-fonte completo



Flutter é um **framework** (ferramenta) de desenvolvimento com foco **multiplataforma em dispositivos móveis**.

Criado pela **Google**, é bastante utilizado no mercado e, mais recentemente, permite a **criação de aplicações para desktop** (Linux, Windows e macOS). Vamos entender mais sobre isso?

Na prática, com o Flutter, você escreve o código uma vez e o aplicativo roda em diferentes plataformas - Android e iOS. Logo, seus principais benefícios são a **versatilidade, menor curva de aprendizado e agilidade**.

A ferramenta é vantajosa para as empresas. Uma pessoa desenvolve o aplicativo para **sistemas operacionais diferentes**, ao contrário do modelo nativo, que necessita de duas pessoas devs, uma para cada plataforma.

Qual linguagem o Flutter utiliza?

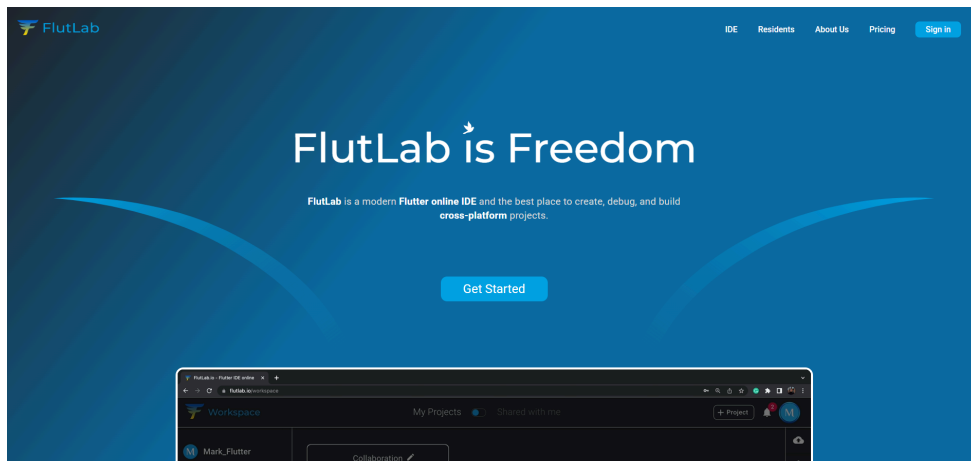
O Flutter utiliza **Dart**, linguagem também criada pela Google e **otimizada para dispositivos clientes** (ou *client-side*) multiplataforma.

O objetivo dela é ser a mais **produtiva** possível e também mais **flexível** para servir como base para diversos frameworks de desenvolvimento de aplicativos.

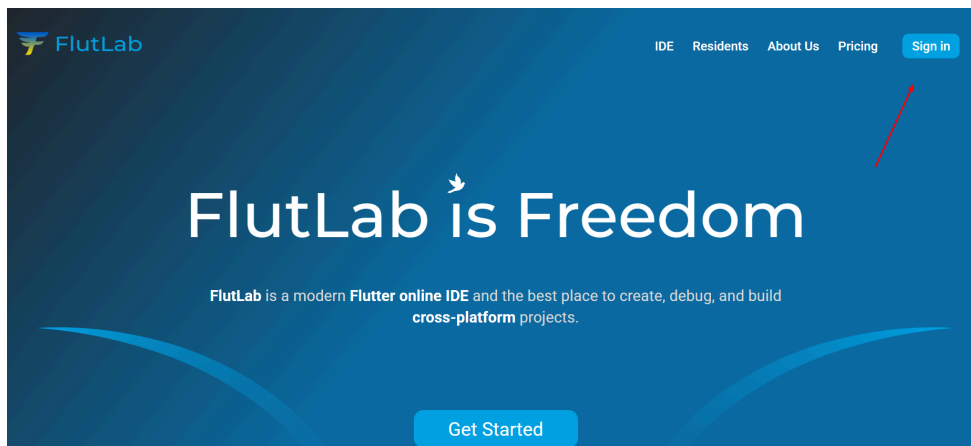
Conheça mais no site oficial: <https://flutter.dev/>

Conhecendo o FlutLab

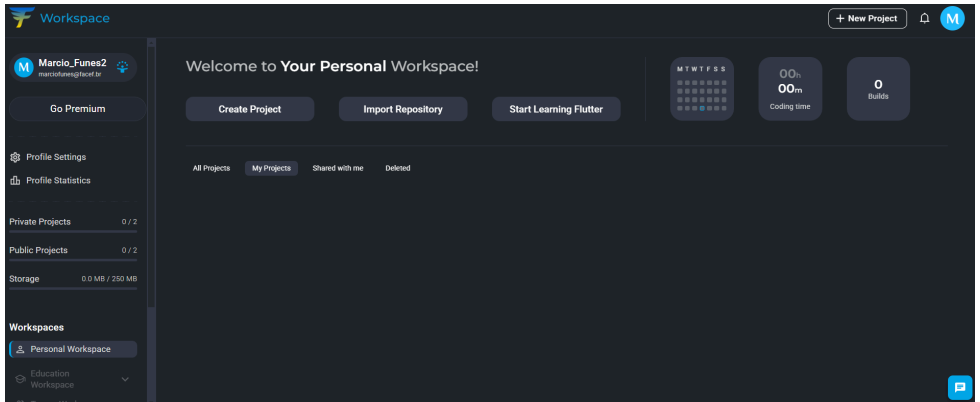
1 - Para começarmos a entender melhor como funciona a estrutura de um projeto mobile com **Flutter**, antes de configurar um ambiente de desenvolvimento local que necessita diversas configurações, vamos utilizar uma ferramenta web chamada **FlutLab**: <https://flutlab.io/>



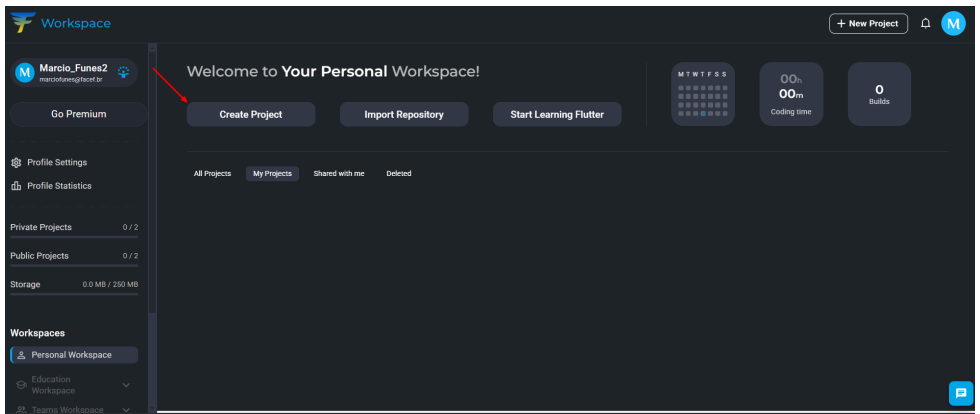
2 - Crie uma conta para iniciarmos o desenvolvimento:



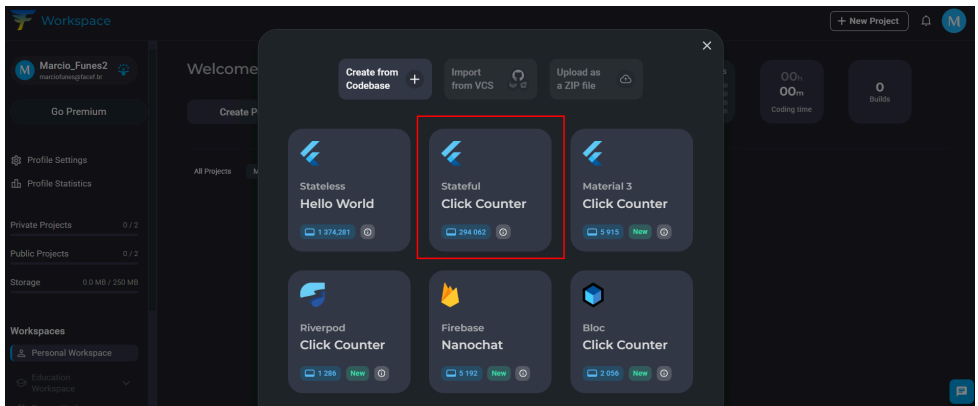
3 - Após o login esse será o workspace para criar e organizar seus apps:



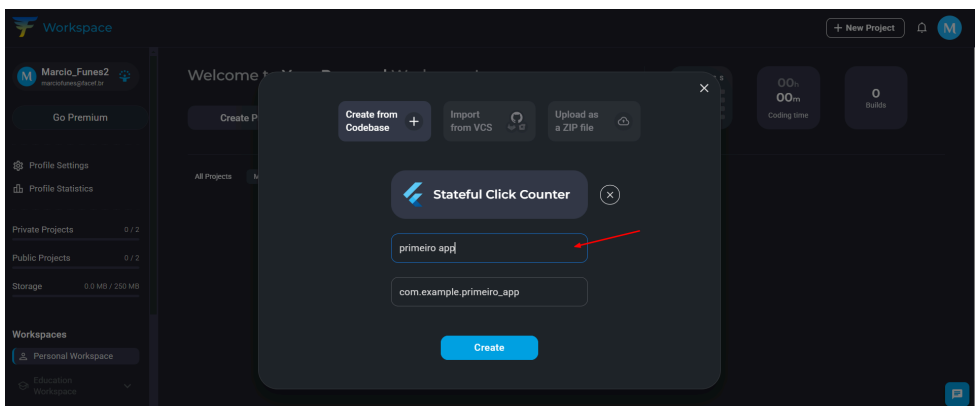
4 - Vamos conhecer a ferramenta. Clique em **Create Project**:



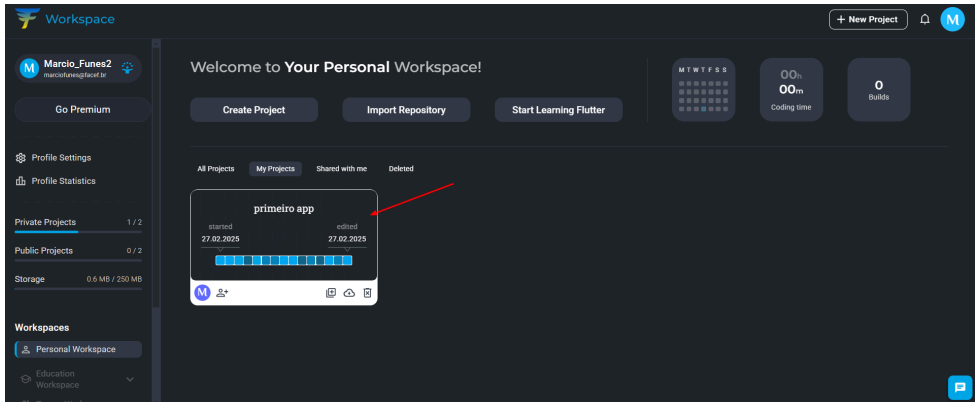
5 - O FlutLab oferece alguns templates para facilitar o início do desenvolvimento. Escolha o template abaixo:



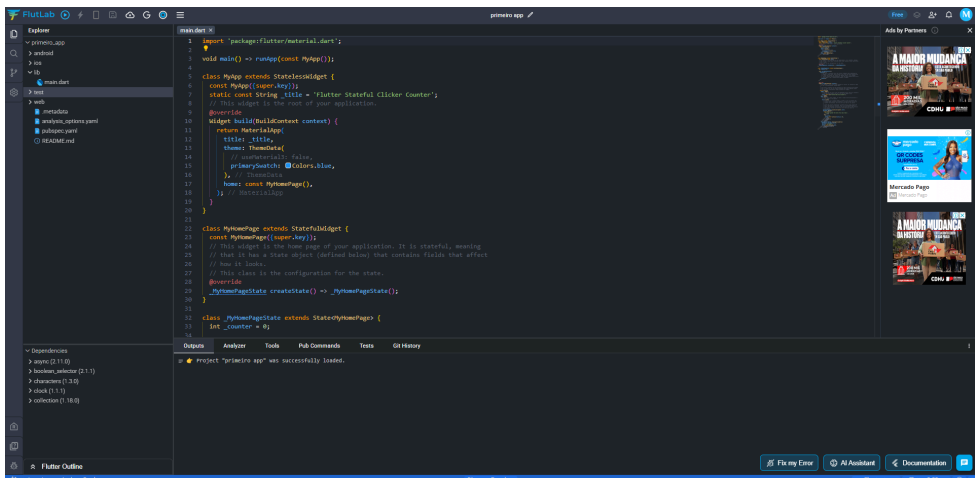
6 - Dê um nome ao seu projeto e clique em **Create**:



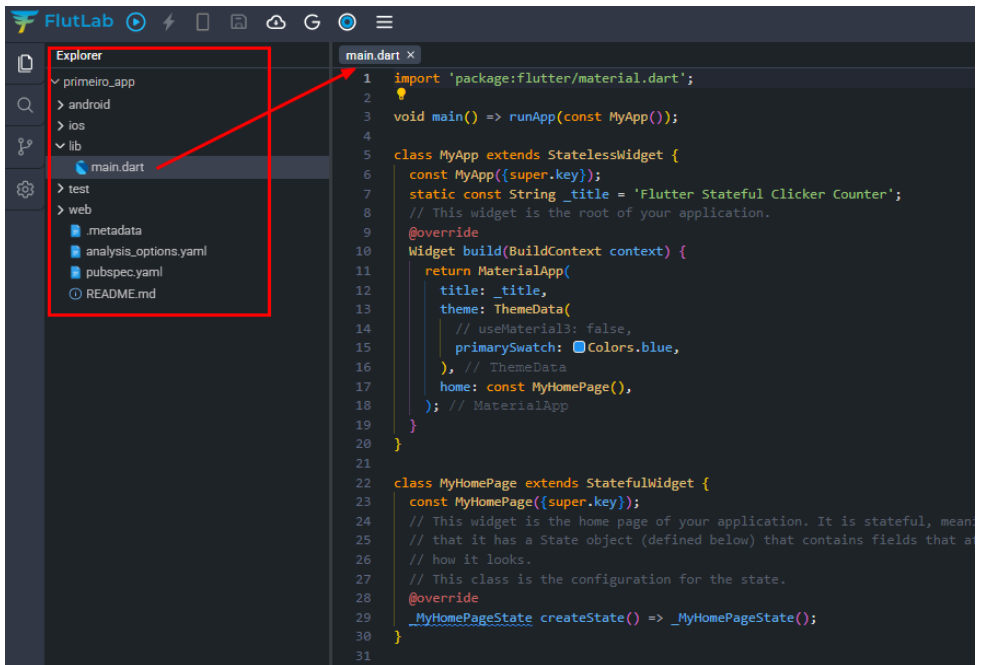
7 - Após o projeto criado, abra o projeto clicando no local abaixo:



8 - Essa é a IDE de desenvolvimento do FlutLab, veja que uma estrutura básica de projeto mobile foi criada. Explore a organização de pasta e arquivos para que você comece a se ambientar:

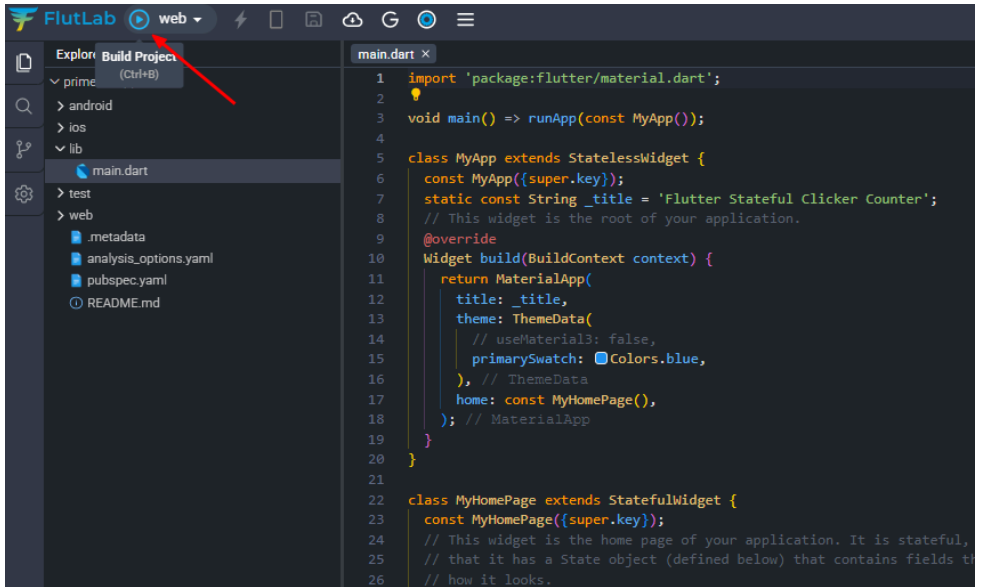


9 - Perceba que diversos arquivos necessários para a composição de uma aplicação mobile já foram criados. Também podemos observar que o **main.dart** foi criado, explore o código para se familiarizar com a linguagem **Dart**.

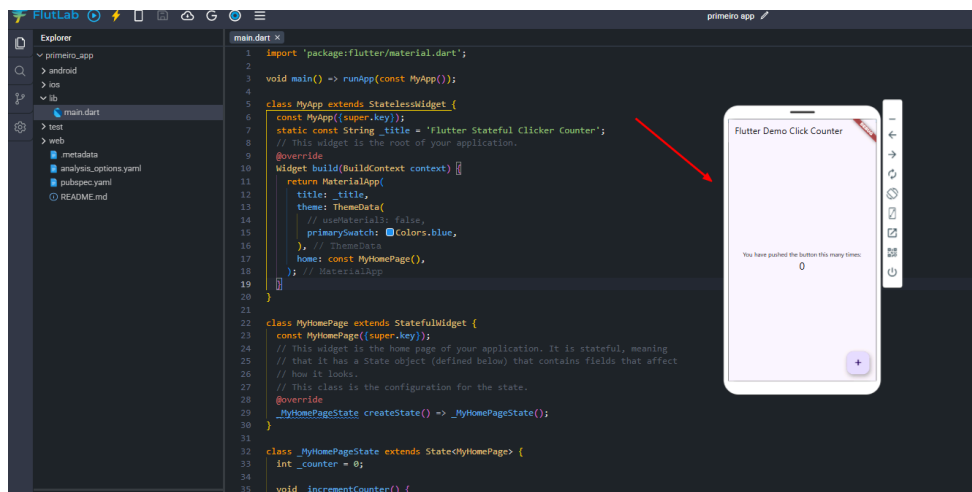


‘Buildando’ uma aplicação Flutter

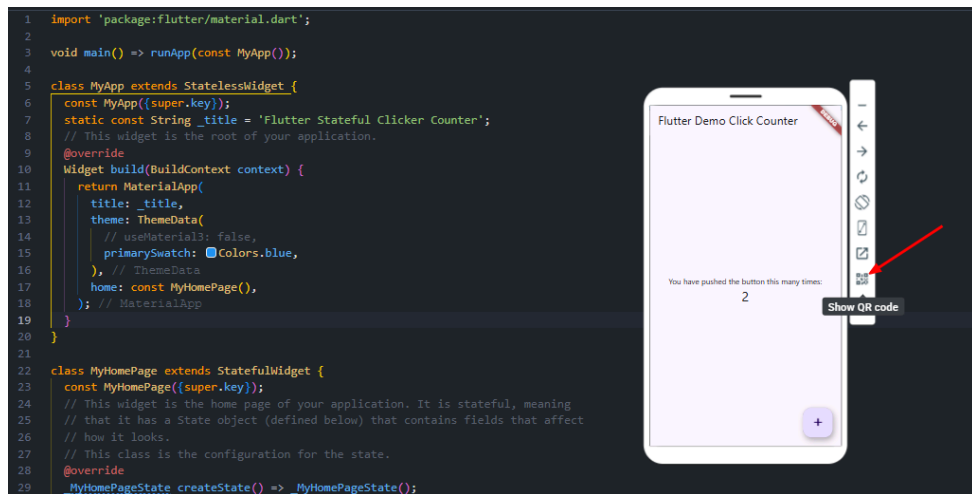
10 - Para testar se existe algum erro, vamos realizar o primeiro build da aplicação. Clique na opção **Build Project** onde indicado pela seta vermelha, ou pressione **CTRL+B** e aguarde a finalização do build.



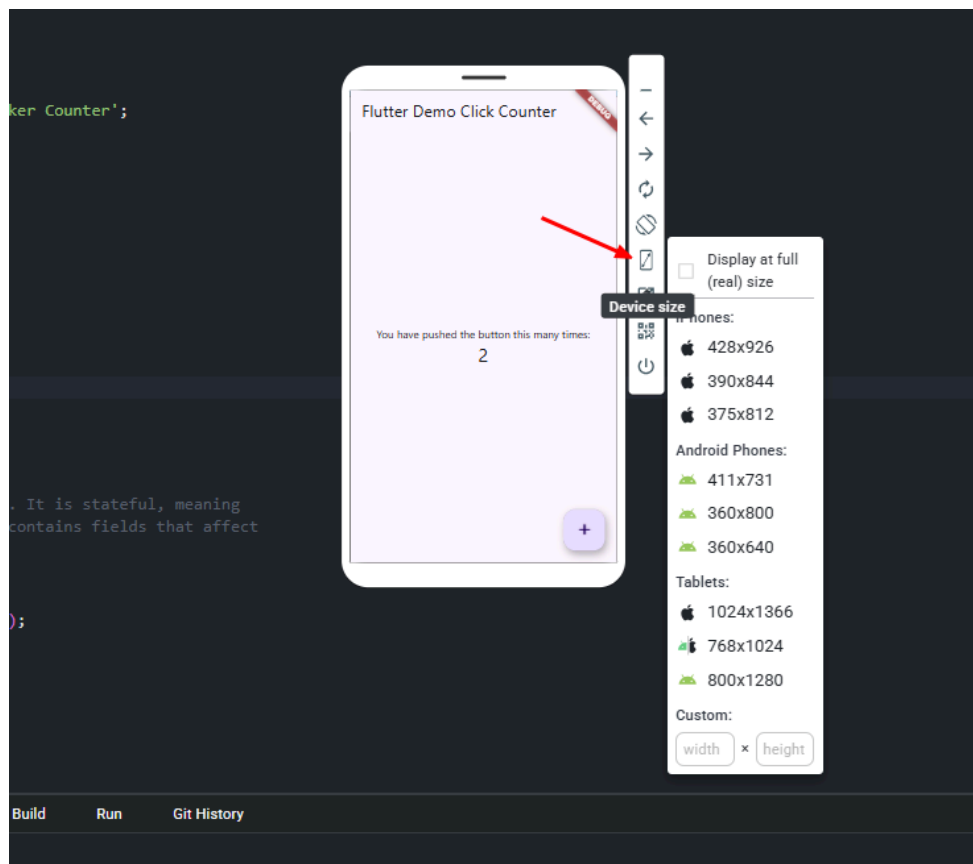
11 - Veja que agora temos uma emulação de um dispositivo mobile, isso ajuda muito durante a visualização das alterações de código realizadas durante o desenvolvimento evitando que tenhamos que gerar e executar o aplicativo em um dispositivo real.



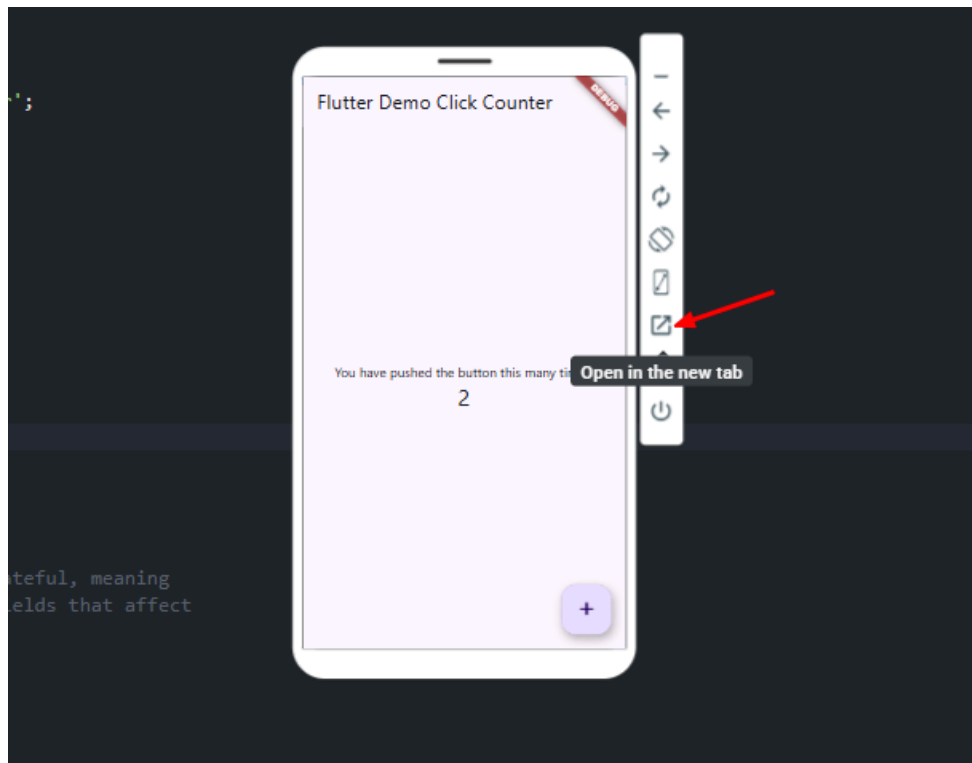
12 - Outra praticidade do FlutLab é testar a aplicação em nosso dispositivo mobile. Clique no ícone **Show QR code** e abra uma pré-visualização da aplicação em seu dispositivo móvel.



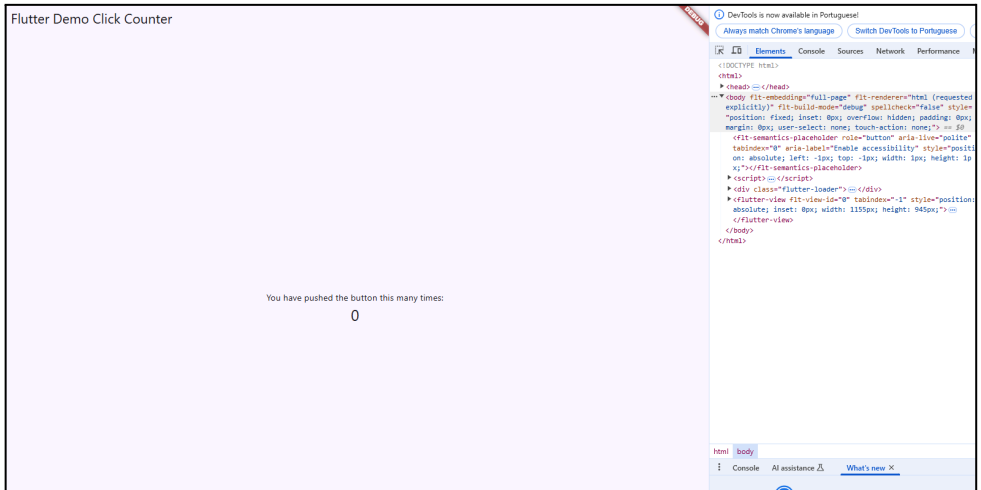
13 - Outro recursos importantes é podermos modificar a resolução da tela e assim testarmos nossa aplicação em diferentes dispositivos.



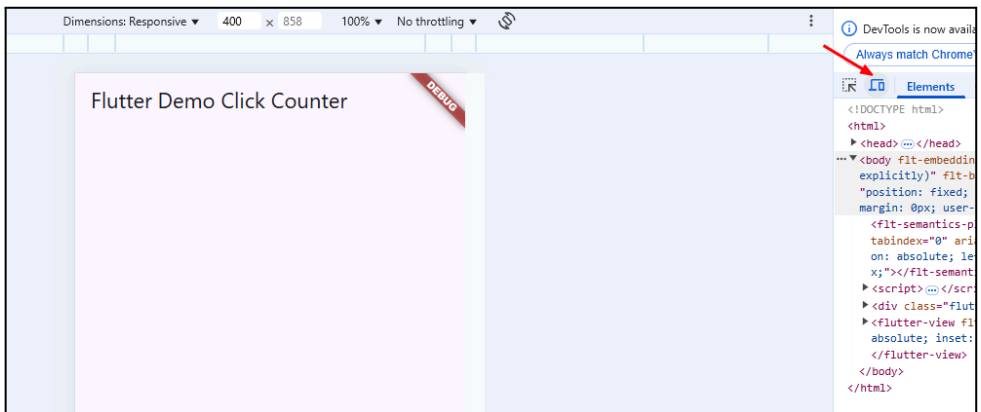
14 - Clique na opção Open in the new tab. Lembre-se que o Flutter possibilita o desenvolvimento de aplicações multiplataforma, ou seja, também podemos utilizar essa mesma aplicação na web.



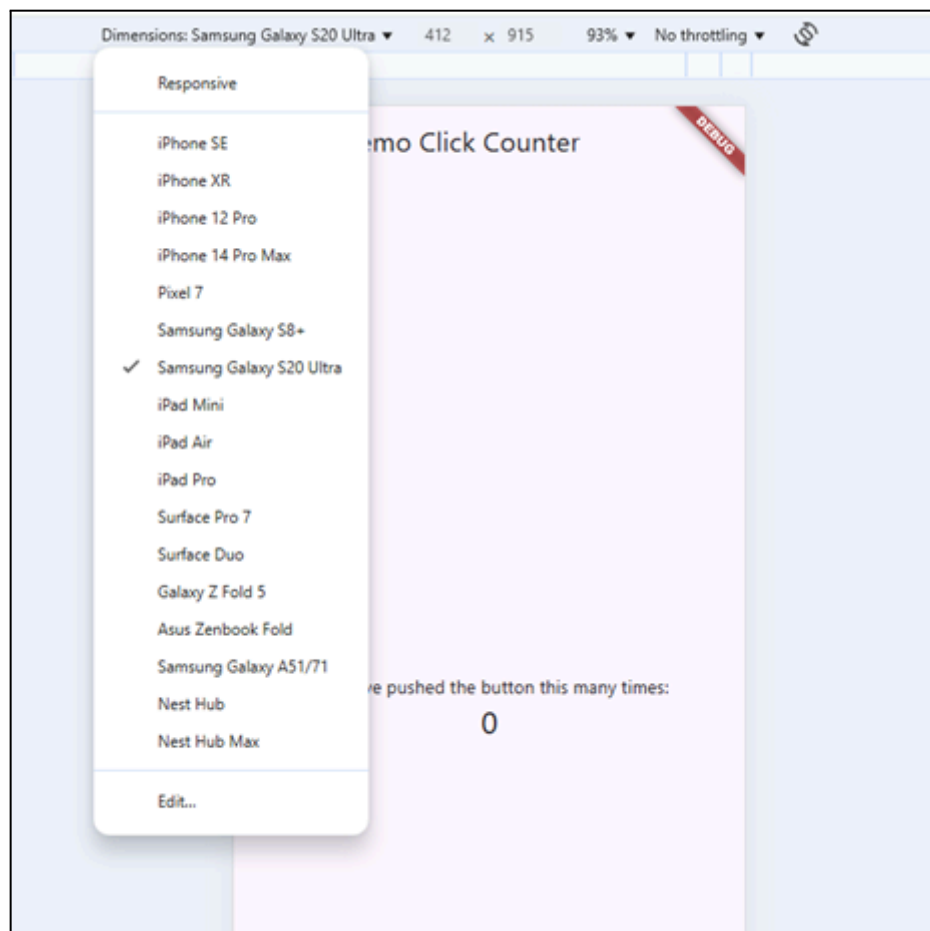
15 - Na nova aba aberta com nossa aplicação, abra as ferramentas de desenvolvimento pressionando F12 (Chrome) ou dê um botão direito na tela e selecione a opção Inspecionar.



16 - Clique na opção Device Toolbar:

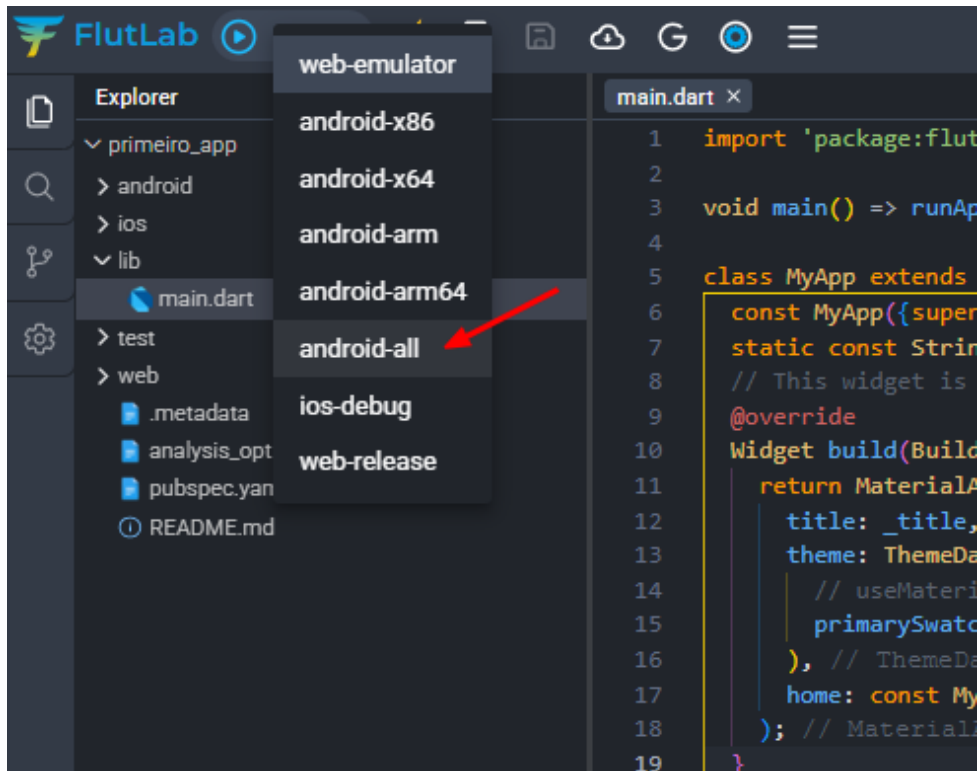


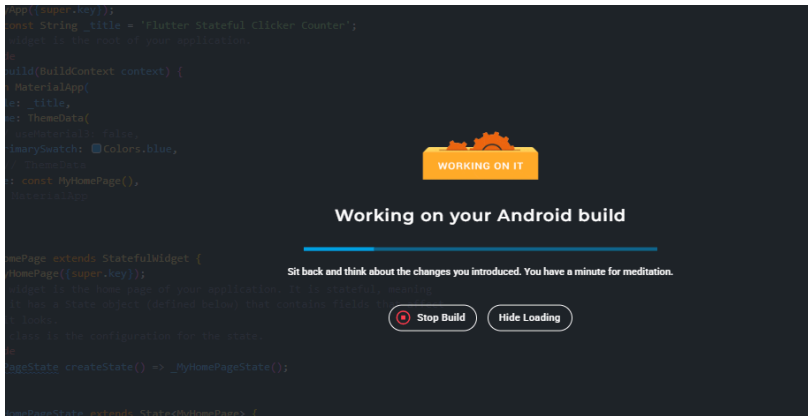
17 - Veja que a aplicação já está totalmente responsiva, ou seja, qualquer dispositivo móvel que abrir essa aplicação web terá o ajuste automático de resolução da tela.



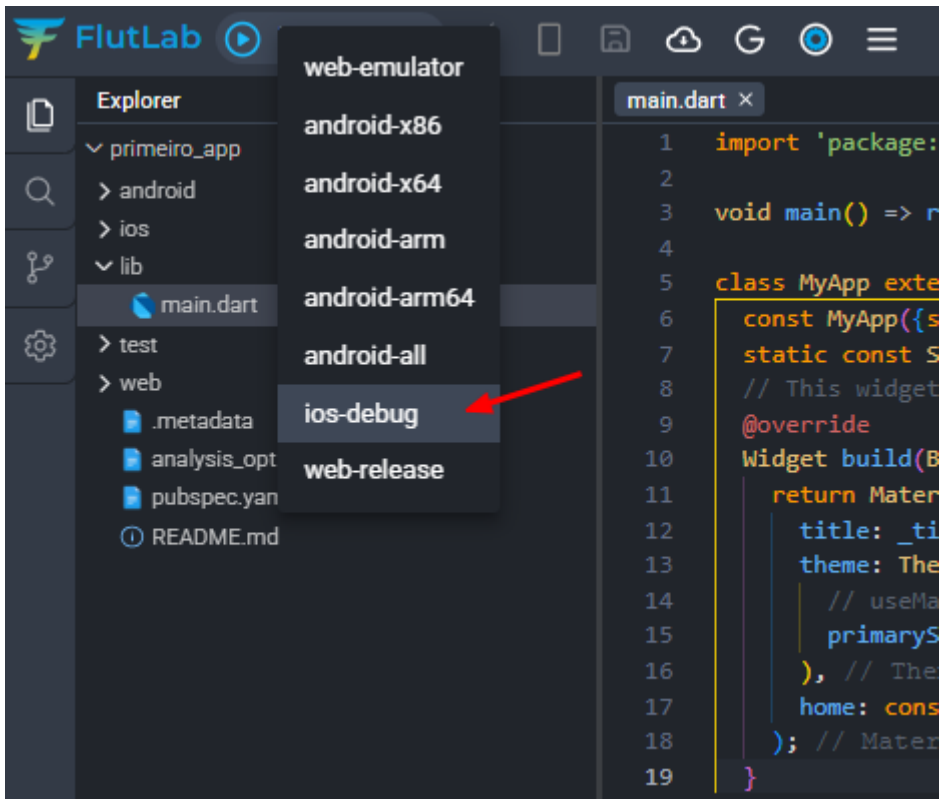
Baixar e testar um .apk no smartphone

(Android) 18 - Testar na web é interessante, mas queremos de fato ver a aplicação rodando após a instalação real em um dispositivo mobile. Volte ao projeto e vamos agora build nosso primeiro **.apk** (extensão de aplicativos mobile para android). Selecione a opção para que a ferramenta crie nosso **.apk** e faça um novo build, aguarde a finalização.

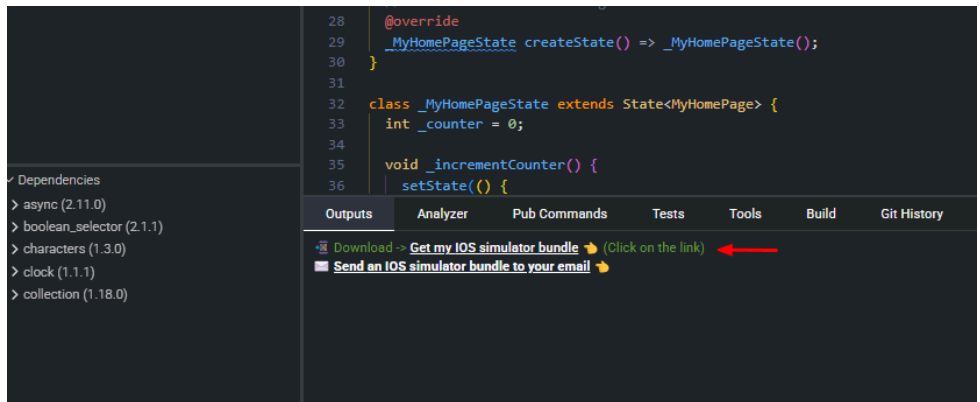




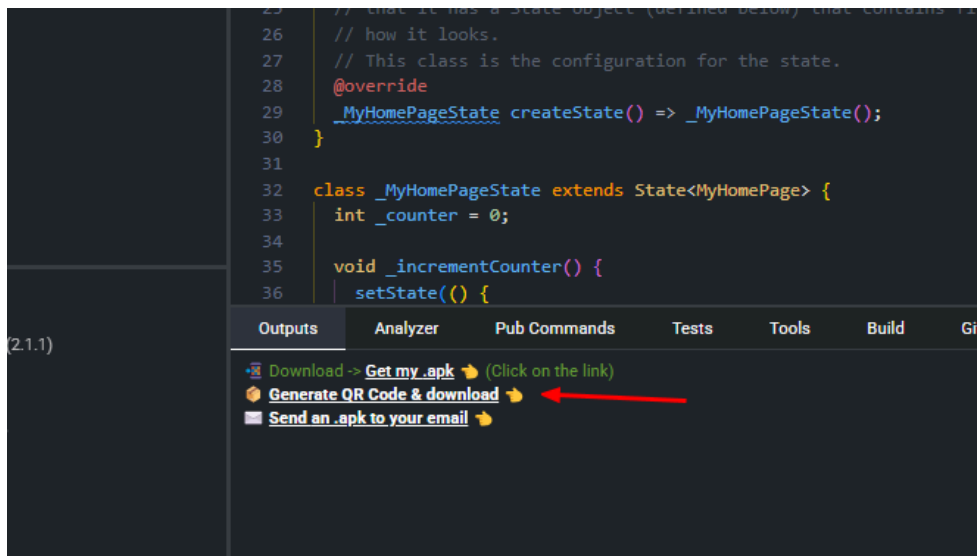
(IOS) 18 - Caso queira testar no IOS escolha a opção **ios-debug** e aguarde o build.



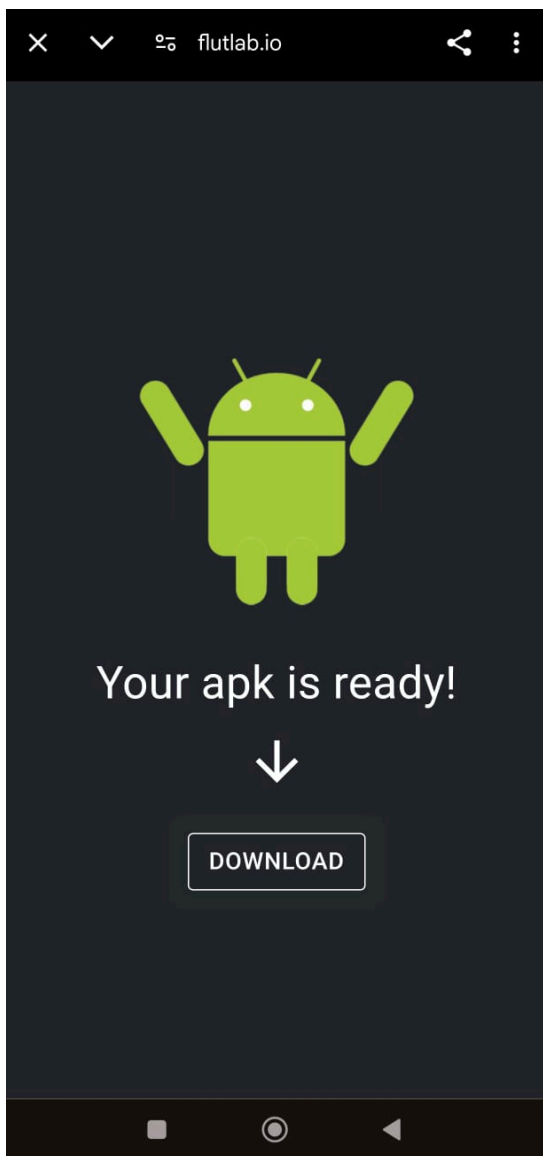
(IOS) 19 - Após o build IOS veja que existem 2 opções para abrir no iPhone, faça um teste utilizando ambas as opções:



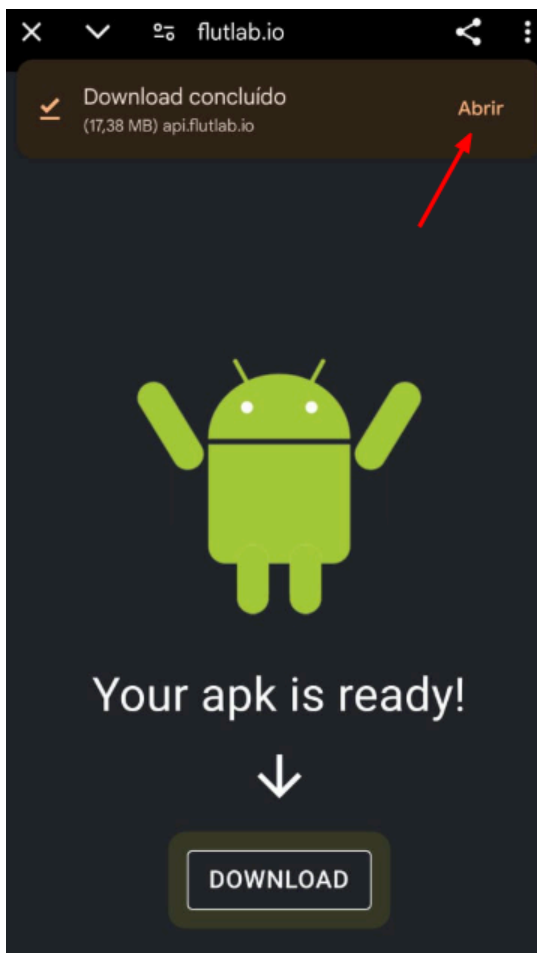
(Android) 20 - Após o build finalizar, clique na opção **Generate QR Code & download** e utilizando seu smartphone android, abra o link do QR Code gerado.



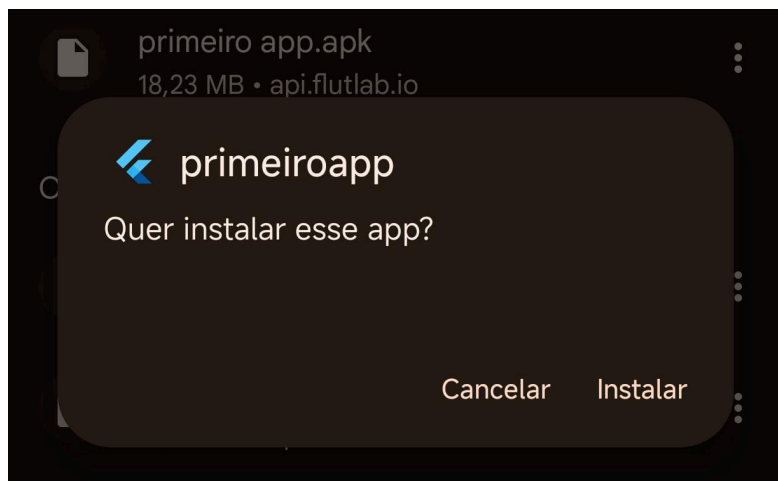
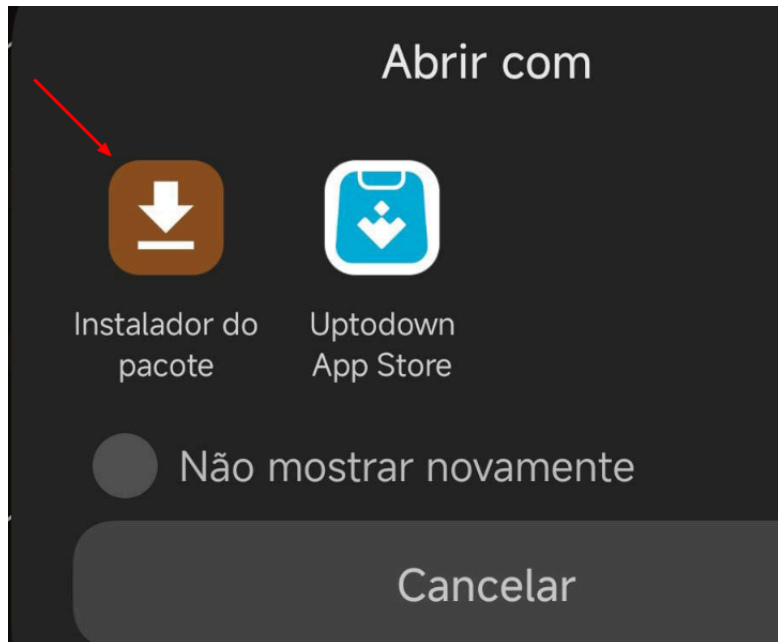
(Android) 20 - Em seu smartphone clique na opção Download e aguarde o download do arquivo .apk.



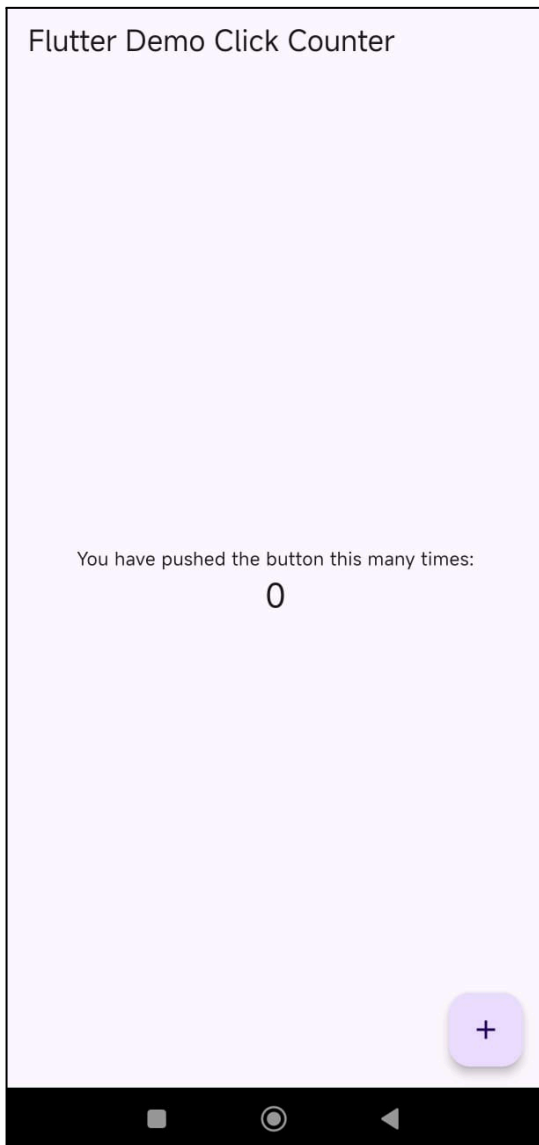
(Android) 21 - Após o download do .apk clique em Abrir:



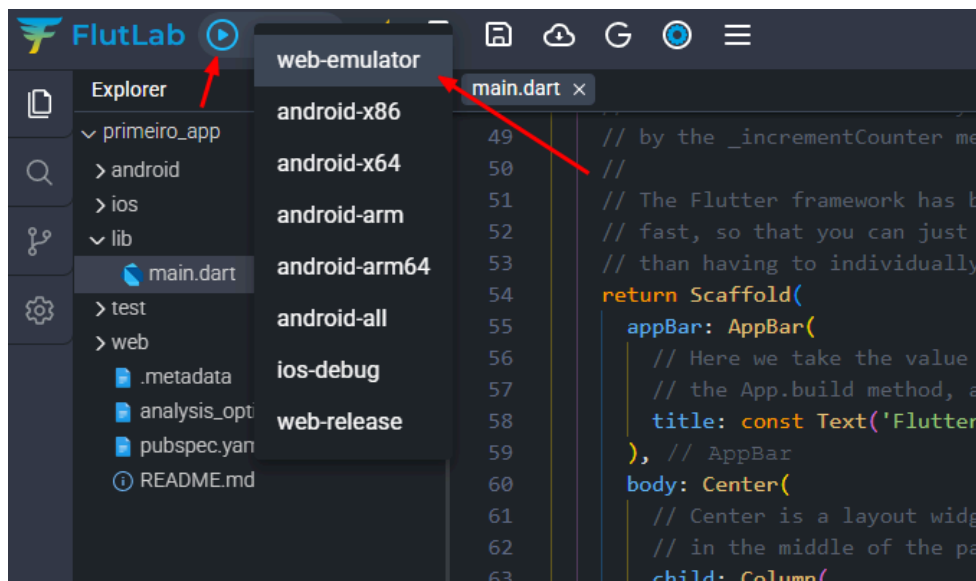
(Android) 22 - Faça a instalação do aplicativo. A depender do smartphone essa tela pode ser diferente.



(Android) 23 - Após a instalação abra seu primeiro aplicativo e teste apertando o botão de + e apenas incrementando o valor na tela:

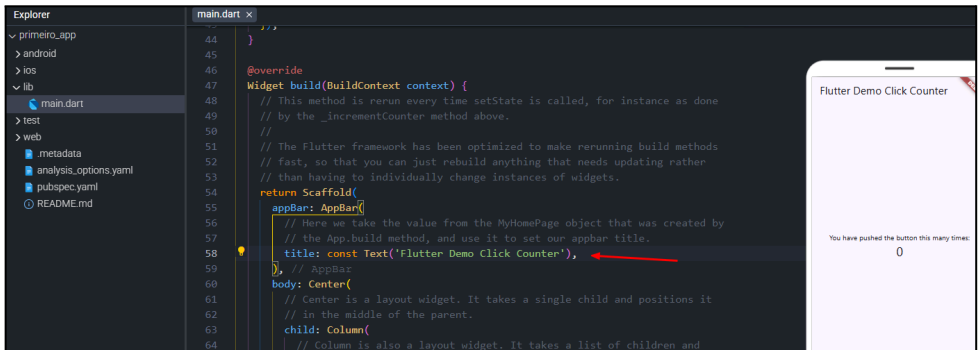


24 - Agora que temos a estrutura básica e já testamos a build da aplicação, vamos fazer algumas alterações para entender como funciona a linguagem Dart. Volte na opção de build e escolha a opção **web-emulator** e clique para realizar o build, aguarde o build finalizar. Com isso podemos ver no emulador de smartphones as alterações que fizemos.

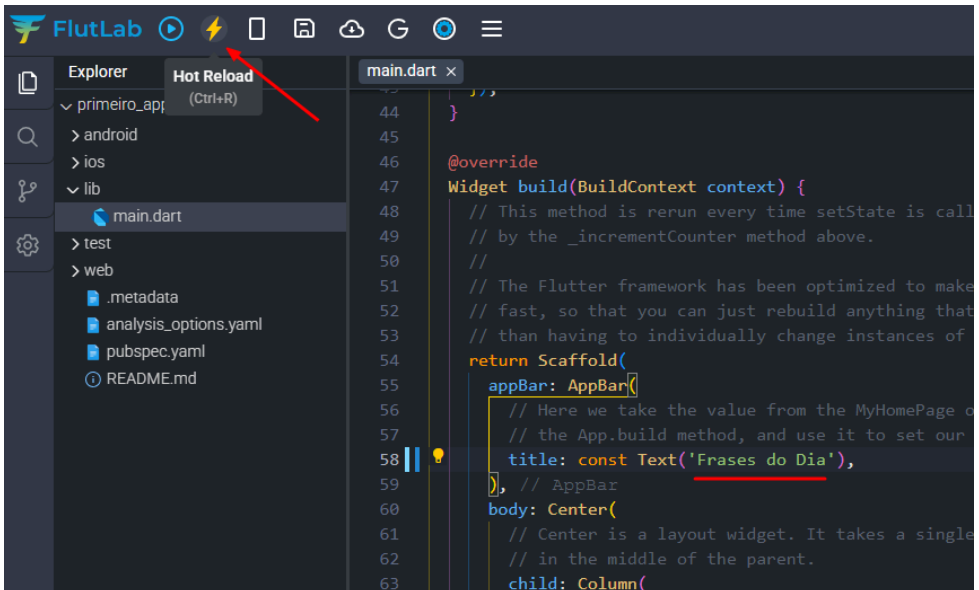


Programando em Dart

25 - Vamos alterar o texto exibido na AppBar, vá até a linha 58 do arquivo main.dart.

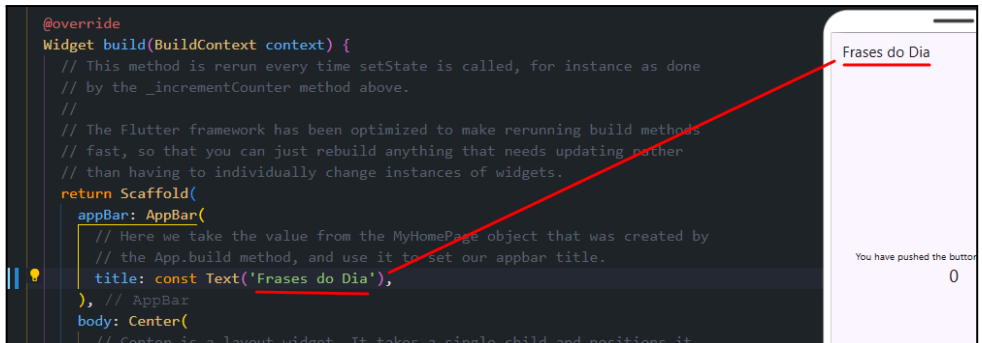


26 - Após alterar o texto clique em **Hot Reload** (CTRL+R):



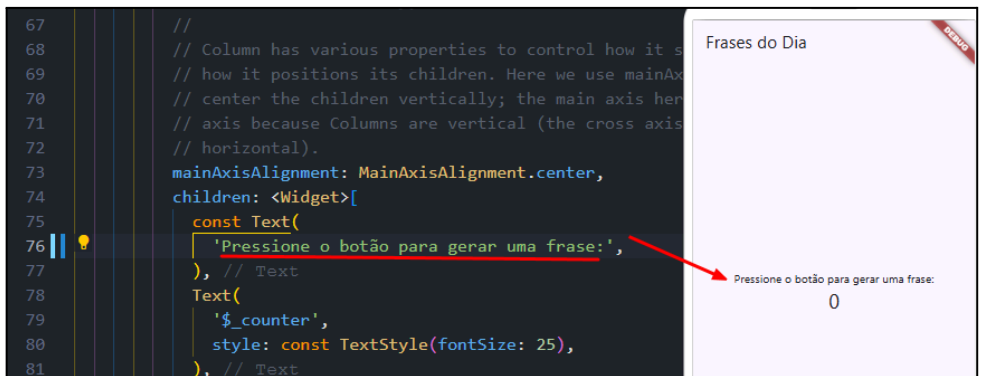
27 - Veja que já podemos visualizar em tempo real as alterações:

```
@override
Widget build(BuildContext context) {
  // This method is rerun every time setState is called, for instance as done
  // by the _incrementCounter method above.
  //
  // The Flutter framework has been optimized to make rerunning build methods
  // fast, so that you can just rebuild anything that needs updating rather
  // than having to individually change instances of widgets.
  return Scaffold(
    appBar: AppBar(
      // Here we take the value from the MyHomePage object that was created by
      // the App.build method, and use it to set our appBar title.
      title: const Text('Frases do Dia'),
    ), // AppBar
    body: Center(
      // Center is a layout widget. It takes a single child and positions it
```

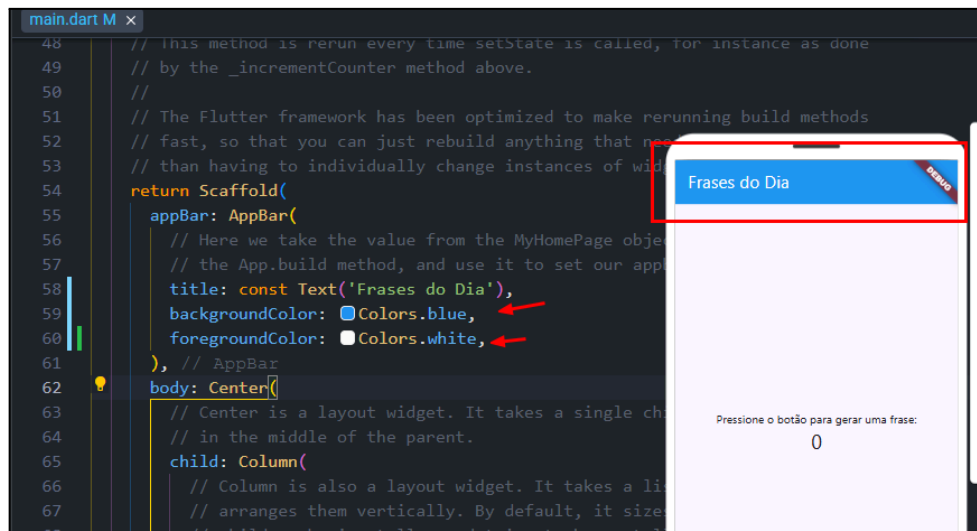


28 - Altere também o texto que aparece na linha 76:

```
67 //
68 // Column has various properties to control how it s
69 // how it positions its children. Here we use mainAx
70 // center the children vertically; the main axis her
71 // axis because Columns are vertical (the cross axis
72 // horizontal).
73 mainAxisAlignment: MainAxisAlignment.center,
74 children: <Widget>[
75   const Text(
76     'Pressione o botão para gerar uma frase:',
77   ), // Text
78   Text(
79     '$_counter',
80     style: const TextStyle(fontSize: 25),
81   ), // Text
```



29 - Vamos alterar as cores na nossa AppBar, o fundo será azul e o texto branco:



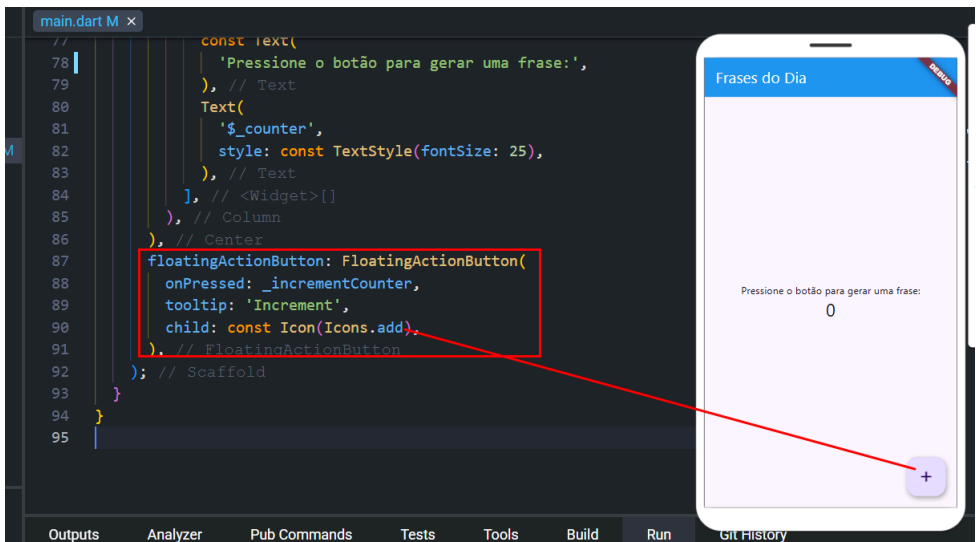
Manipulando um Widget

30 - Vamos manipular, por exemplo, um **widget** existente em nosso código chamado **FloatingActionButton()**.

No Flutter, um **widget** é uma construção que descreve parte da interface de usuário (UI) do aplicativo.

O **FloatingActionButton** é um tipo de **widget** que representa um botão flutuante que geralmente é utilizado para ações primárias no seu aplicativo, como adicionar, editar ou outra ação importante.

Esse widget possui várias propriedades que permitem personalizar seu comportamento e aparência, como **onPressed**, **tooltip**, **child**, entre outras.



31 - Mude o texto da dica ao usuário sobre a funcionalidade do botão (tooltip) e também vamos mudar o ícone na linha 90. Não esqueça de pressionar o botão Hot Reload (CTRL+R) depois de cada alteração no código:



32 - Perceba que na linha 88 temos a propriedade chamada “onPressed” que define o que deverá acontecer quando este botão é pressionado. Veja que ele chama uma função chamada `_incrementCounter`. Vamos alterar o comportamento dessa função, segurando o CTRL clique em `_incrementCounter` para que a IDE exiba essa função (linha 35).

```
85         ), // Column
86     ), // Center
87     floatingActionButton: FloatingActionButton(
88         onPressed: _incrementCounter,
89         tooltip: 'Rock and Roll',
90         child: const Icon(Icons.rocket_launch),
91     ), // FloatingActionButton
92 ); // Scaffold
93 }
94 }
```

33 - Para efeitos de aprendizagem, vamos mudar essa função para ao invés de incrementar um número, ela deverá gerar um número aleatório. Mude o valor da variável “counter” para **_numeroAleatorio**, dentro de **setState** chame essa nova variável e vamos passar como valor uma função do tipo Random.

Veja que o **Dart** não reconhece o nome Random como um função.

```
main.dart M x
25 // that it has a State object (defined below) that contain
26 // how it looks.
27 // This class is the configuration for the state.
28 @override
29 MyHomePageState createState() => MyHomePageState();
30 }
31
32 class MyHomePageState extends State<MyHomePage> {
33   int numeroAleatorio = 0;
34
35   void _incrementCounter() {
36     setState(() {
37       numeroAleatorio = Random;
38     });
39   }
40 }
41
42
```



Importando uma biblioteca no Dart

34 - Para utilizarmos a função Random no Dart, primeiro temos que importar a biblioteca **dart:math**. Faça a importação como abaixo:

```
main.dart M x
1  import 'dart:math';
2
3  import 'package:flutter/material.dart';
4
5  void main() => runApp(const MyApp());
6
7  class MyApp extends StatelessWidget {
8    const MyApp({super.key});
9    static const String title = 'Flutter Stateful Cl
```

35 - Agora conseguimos de fato chamar a função Random de modo a ser reconhecido pelo Dart. Vamos fazer com que a função escolha um número aleatório de 0 até 3.

```
33
34  class _MyHomePageState extends State<MyHomePage> {
35    int numeroAleatorio = 0;
36
37    void _incrementCounter() {
38      setState(() {
39        numeroAleatorio = new Random().nextInt(4); //0,1,2,3
40      });
41    }

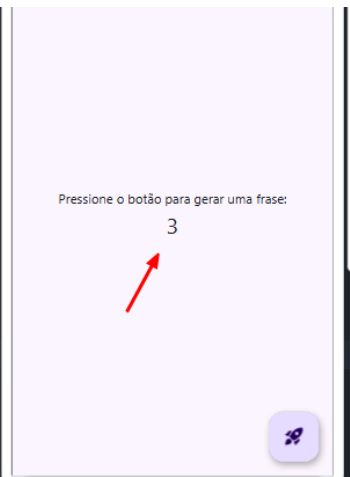
```

36 - Vamos agora dizer ao Dart que queremos que a variável _numeroAleatorio apareça na tela do app como na linha 79:

```
73     children: <Widget>[
74       const Text(
75         'Pressione o botão para gerar uma frase:',
76       ), // Text
77       Text(
78         // '$_counter',
79         '$_numeroAleatorio',
80         style: const TextStyle(fontSize: 25),
81       ), // Text
82     ], // <Widget>[]
83   ), // Column
```

37 - Faça o Hot Reload, veja que agora pressionando o botão geramos os números aleatórios.

```
// horizontal).
mainAxisAlignment: MainAxisAlignment.center,
children: <Widget>[
  const Text(
    'Pressione o botão para gerar uma frase:',
  ), // Text
  Text(
    // '$_counter',
    '$_numeroAleatorio',
    style: const TextStyle(fontSize: 25),
  ), // Text
], // <Widget>[]
), // Column
, // Center
floatingActionButton: FloatingActionButton(
  onPressed: _incrementCounter,
  tooltip: 'Rock and Roll',
```



Customizando nossa primeira aplicação mobile

38 - Mas nosso app deverá gerar frases do dia e não números aleatórios, portanto, vamos criar uma lista de frases, como abaixo:

```
33
34 class _MyHomePageState extends State<MyHomePage> {
35   int _numeroAleatorio = 0;
36   List _frases = [
37     "A vida é o que acontece enquanto você está ocupado fazendo outros planos." – John Lennon',
38     "Faça o que você ama e nunca terá que trabalhar um dia na sua vida." – Confúcio',
39     "O sucesso é ir de fracasso em fracasso sem perder o entusiasmo." – Winston Churchill',
40     "Não é o que você olha que importa, é o que você vê." – Henry David Thoreau'
41 ];
42
43 void _incrementCounter() {
44   setState(() {
```

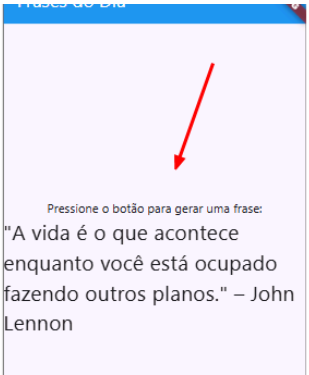
```
List _frases = [
  "A vida é o que acontece enquanto você está ocupado fazendo outros planos." – John Lennon',
  "Faça o que você ama e nunca terá que trabalhar um dia na sua vida." – Confúcio',
  "O sucesso é ir de fracasso em fracasso sem perder o entusiasmo." – Winston Churchill',
  "Não é o que você olha que importa, é o que você vê." – Henry David Thoreau'
];
```


39 - Agora queremos que as frases apareçam na tela, porém, queremos que elas apareçam de modo aleatório. Para isso, na linha 86, chame a lista **_frases** e como a posição do vetor, dentro do colchetes, coloque nossa variável **_numeroAleatorio**.

```
78     mainAxisAlignment: MainAxisAlignment.center,  
79     children: <Widget>[  
80       const Text(  
81         'Pressione o botão para gerar uma frase:',  
82       ), // Text  
83       Text(  
84         // '$_counter',  
85         // '$_numeroAleatorio',  
86         _frases[_numeroAleatorio],  
87       ),  
88       style: const TextStyle(fontSize: 25),  
89     ), // Text  
90   ], // <Widget>[]
```

40 - Apesar de ser formatação, funciona perfeitamente:

```
// horizontal).  
mainAxisAlignment: MainAxisAlignment.center,  
children: <Widget>[  
  const Text(  
    'Pressione o botão para gerar uma frase:',  
  ), // Text  
  Text(  
    // '$_counter',  
    // '$_numeroAleatorio',  
    _frases[_numeroAleatorio],  
  ),  
  style: const TextStyle(fontSize: 25),  
), // Text  
, // <Widget>[]
```



The screenshot shows a mobile application interface. At the top, there is a title bar that says "Frases do Dia". Below it, there is a button with the text "Pressione o botão para gerar uma frase:". Below the button, there is a quote: "A vida é o que acontece enquanto você está ocupado fazendo outros planos." – John Lennon. A red arrow points from the button to the quote.

41 - Podemos alterar alguns estilos de nosso texto:

```
82 ), // Text
83 Text(
84   //'$_counter',
85   //'$_numeroAleatorio',
86   _frases[_numeroAleatorio],
87
88   style: const TextStyle(
89     fontSize: 25,
90     color: Colors.blue, // Cor do texto
91     fontWeight: FontWeight.bold, // Peso da fonte (negrito)
92     fontStyle: FontStyle.italic, // Estilo itálico
93   ), // TextStyle
94 ), // Text
```

Pressione o botão para gerar uma frase:

***"A vida é o que acontece
enquanto você está ocupado
fazendo outros planos." –
John Lennon***

42 - Caso queira, adicione um espaçamento entre textos:

```
78
79 mainAxisAlignment: MainAxisAlignment.center,
80 children: <Widget>[
81   const Text(
82     'Pressione o botão para gerar uma frase:',
83   ), // Text
84   SizedBox(height: 20), //adiciona um espaçamento
85   Text(
86     //'$_counter',
87     //'$_numeroAleatorio',
88     _frases[_numeroAleatorio],
89
90     style: const TextStyle(
91       fontSize: 25,
92       color: Colors.blue, // Cor do texto
93       fontWeight: FontWeight.bold, // Peso da fonte (negrito)
94       fontStyle: FontStyle.italic, // Estilo itálico
95     ), // TextStyle
96   ), // Text
```

Frases do Dia

Pressione o botão para gerar uma frase:

***"A vida é o que acontece
enquanto você está ocupado
fazendo outros planos." –
John Lennon***

Exercícios:

1. Deixe o texto centralizado e responsivo.
2. Abaixo da frase do dia deverá aparecer um emoji.
3. Adicione uma imagem ou gif centralizado antes dos textos no aplicativo.
4. Faça o build e instale o .apk em seu smartphone para testar as funcionalidades acima.

Código-fonte completo main.dart

```
import 'dart:math';

import 'package:flutter/material.dart';

void main() => runApp(const MyApp());

class MyApp extends StatelessWidget {
  const MyApp({super.key});
  static const String _title = 'Flutter Stateful Clicker Counter';
  // This widget is the root of your application.
  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: _title,
      theme: ThemeData(
        // useMaterial3: false,
        primarySwatch: Colors.blue,
      ),
      home: const MyHomePage(),
    );
  }
}

class MyHomePage extends StatefulWidget {
  const MyHomePage({super.key});
  // This widget is the home page of your application. It is stateful, meaning
  // that it has a State object (defined below) that contains fields that affect
  // how it looks.
  // This class is the configuration for the state.
  @override
  _MyHomePageState createState() => _MyHomePageState();
}

class _MyHomePageState extends State<MyHomePage> {
  int _numeroAleatorio = 0;
  List _frases = [
```

"A vida é o que acontece enquanto você está ocupado fazendo outros planos."
– John Lennon',
"Faça o que você ama e nunca terá que trabalhar um dia na sua vida." –
Confúcio',
"O sucesso é ir de fracasso em fracasso sem perder o entusiasmo." – Winston
Churchill',
"Não é o que você olha que importa, é o que você vê." – Henry David Thoreau'
];

```
void _incrementCounter() {  
  setState(() {  
    _numeroAleatorio = new Random().nextInt(4); //0,1,2,3  
  });  
}
```

```
@override  
Widget build(BuildContext context) {  
  // This method is rerun every time setState is called, for instance as done  
  // by the _incrementCounter method above.  
  //  
  // The Flutter framework has been optimized to make rerunning build methods  
  // fast, so that you can just rebuild anything that needs updating rather  
  // than having to individually change instances of widgets.  
  return Scaffold(  
    appBar: AppBar(  
      // Here we take the value from the MyHomePage object that was created by  
      // the App.build method, and use it to set our appbar title.  
      title: const Text('Frases do Dia'),  
      backgroundColor: Colors.blue,  
      foregroundColor: Colors.white,  
    ),  
    body: Center(  
      // Center is a layout widget. It takes a single child and positions it  
      // in the middle of the parent.  
      child: Column(  
        // Column is also a layout widget. It takes a list of children and  
        // arranges them vertically. By default, it sizes itself to fit its  
        // children horizontally, and tries to be as tall as its parent.  
        //  
        // Column has various properties to control how it sizes itself and
```

```
// how it positions its children. Here we use mainAxisAlignment to
// center the children vertically; the main axis here is the vertical
// axis because Columns are vertical (the cross axis would be
// horizontal).
```

```
mainAxisAlignment: MainAxisAlignment.center,
children: <Widget>[
  const Text(
    'Pressione o botão para gerar uma frase:',
  ),
  SizedBox(height: 20), //adiciona um espaçamento
  Text(
    //"$_counter",
    //"$_numeroAleatorio",
    _frases[_numeroAleatorio],

    style: const TextStyle(
      fontSize: 25,
      color: Colors.blue, // Cor do texto
      fontWeight: FontWeight.bold, // Peso da fonte (negrito)
      fontStyle: FontStyle.italic, // Estilo itálico
    ),
  ),
],
),
),
floatingActionButton: FloatingActionButton(
  onPressed: _incrementCounter,
  tooltip: 'Rock and Roll',
  child: const Icon(Icons.rocket_launch),
),
);
}
```